

Lab 6 – Predictive Parsing Table

Aim

To construct a predictive parsing table for a given grammar using FIRST and FOLLOW sets.

Algorithm

1. Start the program.
2. Read the grammar productions.
3. For each production $A \rightarrow \alpha$, compute **FIRST(α)**.
4. For every terminal a in ****FIRST(α)**, place $A \rightarrow \alpha$ in $M[A,a]$.
5. If ϵ belongs to ****FIRST(α)**, then for every terminal b in ****FOLLOW(A)**, place $A \rightarrow \alpha$ in $M[A,b]$.
6. If any table entry contains more than one production, the grammar is not **LL(1)**.
7. Display the predictive parsing table.
8. Stop.

Grammar Used

$E \rightarrow TR$
 $R \rightarrow +TR$
 $R \rightarrow \#$
 $T \rightarrow FY$
 $Y \rightarrow *FY$
 $Y \rightarrow \#$
 $F \rightarrow (E)$
 $F \rightarrow i$

represents **epsilon**.

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char nonTerminals[] = {'E', 'R', 'T', 'Y', 'F'};
    char terminals[] = {'i', '+', '*', '(', ')', '$'};
    char table[5][6][20];
    int i, j;

    for (i = 0; i < 5; i++)
        for (j = 0; j < 6; j++)
            strcpy(table[i][j], "");

    strcpy(table[0][0], "E->TR");
```

```

strcpy(table[0][3], "E->TR");

strcpy(table[1][1], "R->+TR");
strcpy(table[1][4], "R->#");
strcpy(table[1][5], "R->#");

strcpy(table[2][0], "T->FY");
strcpy(table[2][3], "T->FY");

strcpy(table[3][2], "Y->*FY");
strcpy(table[3][1], "Y->#");
strcpy(table[3][4], "Y->#");
strcpy(table[3][5], "Y->#");

strcpy(table[4][0], "F->i");
strcpy(table[4][3], "F->(E)");

printf("\nPredictive Parsing Table\n\n");

printf("%8s", "");
for (j = 0; j < 6; j++)
    printf("%12c", terminals[j]);
printf("\n");

for (i = 0; i < 5; i++) {
    printf("%8c", nonTerminals[i]);
    for (j = 0; j < 6; j++) {
        printf("%12s", table[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Sample Output

Predictive Parsing Table

	i	+	*	(	)	\$
E	E->TR			E->TR		
R		R->+TR			R->#	R->#
T	T->FY			T->FY		
Y		Y->#	Y->*FY		Y->#	Y->#
F	F->i		F->(E)			

Result

Thus the predictive parsing table was constructed successfully for the given grammar.